

ОТЧЁТ

по индивидуальному заданию №1

Архитектура компьютеров
и операционные системы

Сортировочный центр посылок

Вариант 34

Выполнил:

Бакшинский Павел Петрович
студент 2 курса, группа БПИ252

Больше материалов здесь 

1 Задача и модель

Сценарий обработки

Реализована консольная модель на C11/POSIX в одном процессе и потоке. Посылка проходит входную линию, накопитель, сканер, сортировщик, конвейер и выход нужного направления. Накопители ограничены; заполнение следующей очереди блокирует устройство без потери посылки. После исчерпания повторов чтения посылка направляется на ручную обработку.

Параллелизм имитируется циклом по тактам: разгрузка, конвейеры, сортировщики, сканеры, источники. Устройства хранят посылку и время готовности; поступления и ошибки случайны. На направление выделен один конвейер; время перевозки равно $\lceil L/v \rceil$. Выходы разгружаются периодически; доставка учитывается при поступлении в выход. Ручная обработка не моделируется.

Структура `Parcel` хранит номер, направление, число попыток и стадию посылки. Устройство обслуживает одну посылку; очереди соблюдают FIFO. Входная очередь общая, перед конвейерами очереди отдельные. Порядок обновления устройств не позволяет посылке пройти несколько этапов за такт.

2 Интерфейс и запуск

Аргументы задают объём партии, устройства, вместимости, длительности, ошибки, повторы и `seed`; полный список доступен через `--help`. Без аргументов обрабатываются 20 посылок. Консоль показывает такт, номер посылки, событие и итоговую статистику. Журнал дублирует вывод через `open/write/close`.

Параметр `--seed` воспроизводит события в той же среде; `--delay-ms` меняет только скорость показа. Журнал перезаписывается; запись учитывает частичный вывод и `EINTR`.

Команды запуска

```
make
./build/sorting_center --parcels 10 --error-percent 100 \
  --retries 2 --log run.log
make test
```

3 Завершение работы

Остановка и сохранение состояния

После доставки или передачи на ручную обработку всей партии возвращается код 0. `Ctrl+C/SIGTERM` дают коды 130/143 и частичную статистику; память освобождается, журнал закрывается. Сохраняется баланс: запланировано = доставлено + отказы + незавершённые + ещё не созданные. Некорректные аргументы отклоняются с кодом 2.

Обработчик сигнала только устанавливает флаг. Основной цикл выводит итоги и освобождает ресурсы; сигнал также прерывает ожидание в `nanosleep`. Ошибки вывода возвращают код 1.

4 Проверка программы

Результаты тестирования

На macOS, Apple Clang 17 прошли 14 групп тестов в обычном режиме и с `--check`. Из 10 посылок при 0% ошибок доставлены все; при 100% и двух повторах все переданы вручную за 30 чтений. При вместимости 1 обработаны все 20 посылок; пустая партия завершается сразу. Оба сигнала прерывают длительную задержку с сохранением итогов.

По журналу тесты восстанавливают маршрут каждой посылки и проверяют вместимости, направление и длительности. Проверены 35 наборов параметров и переходы кольцевой очереди через конец массива при вместимостях 1, 3 и 7.

5 Исправления и оценка

Исправления и ограничения

По вопросам из `docs/DEBUGGING.md`: два повтора означают три попытки; выходы освобождаются без повторного учёта доставки; очередь сделана кольцевой с операциями $O(1)$, обход учитывает `head`. Полный контроль каждого такта включается через `--check`, перед завершением выполняется всегда. Сложность основного цикла $O(TK + N)$, с проверкой каждого такта $O(T(N + K))$, где T - такты, K - источники и устройства, N - посылки; паузы и ожидание вывода не учтены. Недостаток - перебор пустых тактов.

Обход очереди использует индекс `(head + i) % limit`, иначе после смещения начала проверка читала бы чужие элементы.

6 Использование ИИ

Четыре запроса из `docs/PROMPTS.md`:

1. У меня `retries = 2`, и после второй ошибки посылка уходит на ручную обработку. Правильно ли я понимаю, что нужны первое чтение и ещё два повтора? Подскажи, с чем сравнивать `attempts` перед повтором
2. После удаления первого элемента я сдвигаю всю очередь. Можно ли обойтись без сдвига, оставив обычный массив? Объясни на маленьком примере
3. Переделал очередь в кольцевую. Есть `head` и `count`, но проверка всё ещё обходит `items[i]` от нуля до `count`. После нескольких извлечений она сообщает об ошибке. Что я не учёл в обходе?
4. После каждого такта я обхожу все созданные посылки. Верно ли, что при T тактах и N посылках получается $O(T * N)$? Если полный обход оставить под флагом `--check`, какие проверки нужны в обычном режиме?